

MODULE 03 · DOMAIN 2 - GENERATING AND READING SQL

Spec to SQL

Turning a data model, a mapping document and a functional spec into a query that is right.

DOMAIN 2 of 5 MODULE 3 of 11 PROMPTS 3

What the last module left you with

- ◆ A list of places the document and the code disagree.
- ◆ A short list of things nobody could verify from the files.
- ◆ Answers to some of those, from the people who owned them.
- ◆ You have not written a line of SQL yet. That was deliberate.

BEFORE YOU START

If you skipped reconciliation, everything in this module builds on documents you have not checked.

A query that runs, returns plausible numbers, and is **silently** wrong.

Two ways to do this

THE DEFAULT

One big ask

- Paste all three documents
- Ask for the query
- Get two hundred lines back
- Start debugging what you cannot read

WHAT WE BUILD

Four checked stages

- Declare the grain before any SQL
- Surface what cannot be implemented
- Build joins, verify counts, then add
- Every stage verified before the next

THE TIEBREAKER

Both take about the same wall clock time. Only one of them tells you where it went wrong.

Declare the grain. Before any SQL exists.

Why grain comes first

- ◆ **Grain is what one row means.** One work order. One part at one site. One movement line.
- ◆ Get it wrong and every measure inflates. Sums, counts, averages, all of them.
- ◆ The query still runs. The numbers still look reasonable.
- ◆ Catching it here costs one sentence. Catching it after a stakeholder queries a dashboard costs a great deal more.

THE POINT

Wrong grain is the most expensive mistake in this workflow and the cheapest one to prevent.

Three tables, one multiplication

Labour hours per work order. Looks correct. Is not.

```
01 SELECT wauf.wonum,  
02        SUM(wopr.actlhrs) AS labour_hours  
03 FROM   wauf  
04 JOIN   wopr ON wopr.wonum = wauf.wonum   -- many ops per W0  
05 JOIN   wres ON wres.wonum = wauf.wonum   -- many reservations  
06 GROUP BY wauf.wonum  
07  
08 -- wres was joined for a filter, not for a measure.  
09 -- Every operation row is now repeated once per reservation.  
10 -- A work order with 4 ops and 5 parts reports 20 rows of hours.
```

WHY IT SURVIVES REVIEW

Nothing here is a syntax error. Nothing fails. The hours are five times too high.

What you actually type

No SQL yet. That instruction is the whole prompt.

```
Using the mapping document and data model below, do NOT write
any SQL yet. Answer these four questions first.
```

1. What does exactly one row of the output represent?
State it as a sentence beginning 'One row per ...'
2. List the join path you intend to use, in order. For each
join, state whether it can return more than one row per
row of the driving table.
3. For any join that can fan out, say how you will prevent
it inflating the measures.
4. Which columns form the unique key of the output?

```
Answer in under 200 words. Do not include any SQL.
```

THE LOAD-BEARING QUESTION

Question two forces the fan-out into the open, before it can hide in a join.

Reading the answer

ACCEPT THIS

Specific and hedged

- Names the grain in one sentence
- Flags reservations as one to many
- Proposes pre-aggregation, and says why
- Names the unique key columns

SEND IT BACK

Confident and vague

- 'One row per work order with its details'
- Lists joins with no cardinality
- Says joins are standard, no fan-out
- Skips the unique key entirely

THE TIEBREAKER

A grain declaration that flags no risk on a four table join has not been done.

Make it declare what it **could not** implement.

Every spec contains something unbuildable

- ◆ A rule that names a concept but never gives you the codes.
- ◆ A calculation whose wording could mean two different things.
- ◆ A grain the source data cannot actually support.
- ◆ The default behaviour is to pick one reading and say nothing.

THE POINT

Silence is the problem. A model that guessed and told you has done its job.

What you actually type

Run this before you ask for code, not after.

```
Read the specification below against the data model provided.  
Do not write SQL.
```

```
List every rule in the spec that you could NOT implement  
exactly as written from the columns available. For each one:
```

- Quote the rule verbatim
- State precisely what is missing or ambiguous
- Give the readings you could take, and which you would pick
- State what evidence would settle it

```
Then list any rule you CAN implement but only by assuming  
something the spec does not state. Name the assumption.
```

```
If you find none, say so explicitly. Do not invent items.
```

WHY THE LAST LINE EXISTS

The last line matters. Without it a model finds problems to fill the shape you asked for.

What it returned on our sample spec

Four items, from a spec that reads as complete

- ◆ **Movement type codes.** Spec names a register held by another team. Not provided.
- ◆ **Excluded part types.** Obsolete parts and kit headers named as concepts, not codes.
- ◆ **Shortfall grain.** Compared at stock location, but the maximum is only held at site.
- ◆ **Currency.** Listed as an open item. Values are in transaction currency.
- ◆ The last two are in the spec. The model found them by reading it properly.

THE RESULT

Two things you cannot build, and two you can build only if you accept a stated compromise.

Build it in stages. Verify between each one.

Four stages, four checks

01

Joins

Skeleton only. Count rows and compare to the driving table.

02

Filters

Add the WHERE clause. Count again. Explain the drop.

03

Transforms

Add derived columns. Spot check twenty rows by hand.

04

Aggregate

Group and sum. Reconcile one part against the source.

IN PRACTICE

Each stage is a separate prompt, and each one ends with a number you look at.

What you actually type, at stage one

Repeat the shape for each stage, changing only what is added.

We are building this query in four stages. This is stage one.

Write the JOIN skeleton only. No WHERE clause, no derived columns, no aggregation. Select only the join keys.

The declared grain is: <paste the grain sentence from prompt 6>

Also give me a COUNT query I can run against this skeleton, and tell me what row count you expect relative to the driving table. If the counts differ, say which join causes it.

Do not proceed to stage two. Wait for the counts.

THE LINE THAT CONNECTS THE PROMPTS

Pasting the grain sentence back in is what keeps stage one honest about stage zero.

What you are checking between stages

- ◆ **After joins.** Row count against the driving table. Any increase is a fan-out.
- ◆ **After filters.** The drop, and whether you can account for it.
- ◆ **After transforms.** Twenty rows, read by eye, against the source.
- ◆ **After aggregation.** One part, one site, reconciled by hand.

THE POINT

Four numbers. If you cannot explain one of them, stop and find out why before continuing.

When it goes wrong mid-build

WHAT MOST PEOPLE DO

Keep iterating

- Paste the error back in
- Get a revision
- Paste the next error back in
- The context fills with dead ends

WHAT WORKS

Restart the stage

- Take what you learned
- Add it to the original prompt
- Start that stage clean
- Earlier stages stay verified

THE TIEBREAKER

Multi-turn debugging degrades. Staged building means restarting costs you one stage, not the query.

You ask for a grain declaration on a five table join. The answer comes back in ninety words, names the grain clearly, lists all four joins, and reports no fan-out risk on any of them.

The query has not been written yet. You have twenty seconds to decide whether to proceed.

What do you do?

- ☐ A Proceed to stage one. The declaration is clear and complete
- ☐ B Ask it to justify the no fan-out claim for each join individually
- ☐ C Ignore it and check the cardinality yourself in the data model
- ☐ D Ask for the query and check the row counts afterwards

◆ PAUSE AND DECIDE

What do you do?

- A

 Proceed, it is complete
Clear is not the same as correct. Four joins with zero risk is the answer to be suspicious of.
- C

 Check cardinality yourself
Correct, and you should. But it is the slower move and B often surfaces the problem in one prompt.
- B

 Ask it to justify each join
Cheapest challenge available. A model asked to defend a claim per join often revises it.
- D

 Get the query, check counts after
This abandons the technique. Row counts after the fact tell you something is wrong, not where.

THE SIGNAL

A clean answer on a complex join is a signal to push, not a signal to proceed.

The unimplementable rules prompt returns one item. A consumption rule depends on movement type codes that live in a register owned by another team.

That team will take three days to respond. Your deadline is Thursday. The deployed legacy procedure contains a list of sixteen codes.

What is the right move?

- ☐ A Copy the sixteen codes from the legacy procedure and proceed
- ☐ B Copy them, proceed, and mark the query as unverified pending the register
- ☐ C Stop work on that measure until the register arrives
- ☐ D Ask the model to infer the correct codes from the movement data

◆ PAUSE AND DECIDE

What is the right move?

- A

 Copy and proceed
Reasonable engineering, but it launders an unverified list into a verified looking query.
- B

 Copy, proceed, mark unverified
Keeps the deadline and keeps the uncertainty visible until the register settles it.
- C

 Stop until the register arrives
Safest and usually wrong. Three days of blocked work to avoid a risk you can carry openly.
- D

 Infer from the data
The data shows which codes exist, never which ones the business counts as consumption.

THE SIGNAL

The legacy list is evidence, not authority. Using it is fine. Forgetting that is not.

At stage one the join skeleton returns 4.1 million rows. The driving table has 890 thousand. The model predicted the counts would match.

You have the four table join in front of you and the data model beside it.

What is the most useful next prompt?

- ☐ A Tell it the counts differ and ask it to fix the query
- ☐ B Ask which join causes the increase, and to prove it with a count per join
- ☐ C Add DISTINCT and move to stage two
- ☐ D Restart stage one with the actual counts included in the prompt

◆ PAUSE AND DECIDE

What is the most useful next prompt?

- A

 Say it differs, ask for a fix
Starts the iteration loop we want to avoid. You get a revision with no diagnosis attached.
- B

 Ask which join, prove with counts
Turns a symptom into a located cause, and the counts are checkable rather than asserted.
- C

 Add DISTINCT, move on
The most damaging option here. It hides the fan-out and the measures still inflate later.
- D

 Restart with the counts
Sound instinct, wrong moment. Restart when iteration has degraded, not on the first surprise.

THE SIGNAL

Stage one exists so that a count mismatch has only four joins to hide in.

Stage four is done. Aggregated totals reconcile against the source for the part you checked by hand. The query is 180 lines and you did not write any of it.

A colleague asks whether it is ready to hand to the reporting team.

What is still missing?

- ☐ A Nothing. Four stages verified and a hand reconciliation is enough
- ☐ B A second reconciliation on a part with different characteristics
- ☐ C A performance review before it runs at full volume
- ☐ D A record of the assumptions carried in from prompt seven

◆ PAUSE AND DECIDE

What is still missing?

- A

 Nothing is missing
One reconciled part proves the query works for that part. It proves very little else.
- B

 A second reconciliation
You should, and pick a part with returns and multiple locations. Still not the biggest gap.
- C

 A performance review
Fair, and it comes later. A slow correct query is a smaller problem than a fast undocumented one.
- D

 A record of the assumptions
Two unverified inputs are baked into this query and nothing on the query says so.

THE SIGNAL

You just spent a module preventing silent decisions. Do not ship one at the last step.

Take this to work tomorrow

- ◆ Next query you are asked for, write the grain sentence yourself before you prompt.
- ◆ Run the unimplementable rules prompt on the spec. It takes two minutes.
- ◆ Build in four stages. Look at the count after each one.
- ◆ Whatever came back from prompt seven goes in a comment at the top of the query.

THE PAYOFF

The last one is thirty seconds of typing and it is the only part that outlives you.

Three prompts, and not one of them asked for the whole query.